AI-POWERED RETAIL SHELF MONITORING AND STOCK AVAILABILITY DETECTION

PHASE I REPORT

Submitted by

ARAVINTHAA S 230701032
KARNEESH D 230701141

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING





RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY: CHENNAI 600025

October 2026

ANNA UNIVERSITY

BONAFIDE CERTIFICATE

Certified that this project report titled "**AI-POWERED RETAIL SHELF MONITORING AND STOCK AVAILABILITY DETECTION**" is the bonafide work of **ARAVINTHAA S (230701032), KARNEESH D (230701141)** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form a part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. J. Manoranjini,

Professor and Head,

Department of Computer Science and Engineering,

Rajalakshmi Engineering College,

Chennai – 602 105.

SIGNATURE

Dr. P. Kumar,

Professor,

Department of Computer Science and Engineering,

Rajalakshmi Engineering College,

Chennai – 602 105.

Submitted for the Viva-Voce Examination held on _____

Internal Examiner External Examiner

ABSTRACT

Retail stores require accurate and timely information about the physical availability of products on shelves. Manual shelf inspection requires continuous human effort and may result in delayed identification of empty or low-stock conditions. Conventional inventory and Point-of-Sale systems primarily record transactional information, which may not always reflect the actual physical condition of products on retail shelves. Therefore, an automated vision-based approach can assist in monitoring shelf conditions and identifying stock availability from shelf images. This project proposes ShelfAI, an AI-powered retail shelf monitoring and stock availability detection system that uses computer vision and object detection to analyse shelf images. In Phase I, a shelf image is uploaded through the application and processed using a trained YOLO11n object detection model. The model identifies visible product-object instances and generates bounding boxes, class information, and confidence scores. The detected objects are counted and analysed within predefined shelf regions. The application then compares the detected quantity with configured minimum-stock thresholds to classify the shelf or product condition as NORMAL, LOW-STOCK, or OUT-OF-STOCK. The system also stores scan results in a database and presents the analysed information through a dashboard, enabling users to monitor shelf conditions and review previous inspections. The Phase I model was trained and evaluated using the SKU110K resized dataset. The implemented YOLO11n model achieved a Precision of 76.4%, Recall of 68.1%, F1-Score of 71.9%, mAP@0.5 of 76.1%, and mAP@0.5:0.95 of 43.7% on the available validation data. An observed inference time of approximately 81.1 ms per image was obtained in the available CPU-based environment. These results demonstrate the feasibility of using vision-based object detection for automated retail shelf monitoring. However, the current dataset and model use a single generic object class and therefore do not provide individual brand or SKU-level product recognition. Phase II will extend the system through a project-specific product dataset and IoT-based physical stock verification using components such as ESP32, load cells, and HX711, enabling more comprehensive physical stock estimation and multimodal verification.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E,** our respected Chairperson **Dr.(Mrs.)THANGAM MEGANATHAN, Ph.D.,** and our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S. N. MURUGESAN, M.E., Ph.D.,** our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J. MANORANJINI, Ph.D.,** Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work.

We convey our sincere and deepest gratitude to our internal guide, **Dr.P.KUMAR,** Professor, Department of Computer Science and Engineering, for his valuable guidance throughout the course of the project. We thank our Project Coordinator, **Dr.S.MADHUSUDHANAN,** Associate Professor, Department of Computer Science and Engineering, for his useful tips during our reviews to build our project.

ARAVINTHAA S

KARNEESH D

CHAPTER NO.	TITLE	PAGE NUMBER
	ABSTRACT	iii
	ACKNOWLEDGEMENT	iv
	TABLE OF CONTENTS	vi
	LIST OF TABLES	xi
	LIST OF FIGURES	xii
	LIST OF ABBREVIATIONS	xiii
1	**INTRODUCTION**	1
	1.1 General	1
	1.2 Problem Statement	2
	1.3 Motivation	2
	1.4 Objectives	3
	1.5 Scope	4
	1.6 Significance of the Project	5
2	**LITERATURE SURVEY**	6
	2.1 Retail Product Recognition	6
	2.2 Dense Shelf Detection and Product Recognition	6
	2.3 Retail Datasets	6
	2.4 Deep Learning for Retail Product Recognition	7
	2.5 Empty-Shelf and Out-of-Stock Detection	8
	2.6 Shelf Monitoring and Planogram Compliance	8
	2.7 Object Detection Models	9

CHAPTER NO.	TITLE	PAGE NUMBER
	2.8 Product Classification and Data Augmentation	9
	2.9 Research Gap	10
3	**PROPOSED SYSTEM**	12
	3.1 System Overview	12
	3.2 Proposed System Workflow	13
	3.3 Major System Modules	13
	3.3.1 User Interface and Web Application Module	13
	3.3.2 Shelf Image Upload Module	14
	3.3.3 Image Preprocessing Module	14
	3.3.4 YOLO11n Object Detection Module	14
	3.3.5 Confidence Filtering Module	14
	3.3.6 Detection Processing Module	15
	3.3.7 Visual Object Counting Module	15
	3.3.8 Shelf Region Assignment Module	15
	3.3.9 Rule-Based Stock Evaluation Module	15
	3.3.10 Stock Status Module	16
	3.3.11 Alert Generation Module	16
	3.3.12 Database and Storage Module	16
	3.3.13 Dashboard and Historical Monitoring Module	16
	3.4 Proposed System Advantages	17
4	**PROPOSED METHODOLOGY AND SYSTEM DESIGN**	18

CHAPTER NO.	TITLE	PAGE NUMBER
	4.1 Introduction	18
	4.2 Phase I System Architecture	18
	4.3 Image Acquisition and Input	19
	4.4 Image Preprocessing	19
	4.5 YOLO11n Object Detection	20
	4.6 Confidence Filtering	20
	4.7 Detection Processing	21
	4.8 Visual Object Counting	21
	4.9 Shelf Region Assignment	22
	4.10 Rule-Based Stock Evaluation	22
	4.11 Stock Status Generation	22
	4.12 Alert Generation	23
	4.13 Database and Storage	23
	4.14 Dashboard and Historical Result Presentation	23
	4.15 Algorithmic Workflow of Phase I	24
	4.16 Algorithms Used in the Proposed System	24
	4.17 Phase I Limitations and Phase II Extension	25
5	**SYSTEM REQUIREMENTS AND DATASET SPECIFICATION**	26
	5.1 Introduction	26
	5.2 Hardware Requirements	26
	5.3 Software Requirements	27

CHAPTER NO.	TITLE	PAGE NUMBER
	5.4 Development Environment	28
	5.5 Dataset Specification	28
	5.6 Dataset Characteristics	30
	5.7 Dataset Preparation and Annotation	30
	5.8 Model Training Specification	30
	5.9 Database Requirements	31
	5.10 Performance Evaluation Metrics	31
	5.11 Application-Level Stock Evaluation	33
6	**TESTING**	35
	6.1 Introduction	35
	6.2 Testing Strategy	35
	6.3 Functional Test Cases	36
	6.4 AI Model Testing and Evaluation	38
	6.5 Integration Testing	39
	6.6 Stock Status and Rule-Based Testing	40
	6.7 Alert Testing	41
	6.8 Database and Dashboard Testing	42
	6.9 End-to-End Testing	42
	6.10 User Acceptance Testing	43
	6.11 Testing Summary	43
7	**RESULTS AND DISCUSSION**	45

CHAPTER NO.	TITLE	PAGE NUMBER
	7.1 Introduction	45
	7.2 Phase I Implementation Result	45
	7.3 YOLO11n Model Evaluation Results	47
	7.4 Visual Detection Results	48
	7.5 Object Counting Results	48
	7.6 Shelf Region and Stock Status Results	48
	7.7 Dashboard and Monitoring Results	49
	7.8 Alert and Monitoring Results	49
	7.9 Database and Historical Monitoring Results	49
	7.10 Sample Application Scan Results	50
	7.11 Discussion of Results	50
	7.12 Overall Phase I Outcome	51
	7.13 Limitations Identified from the Results	52
8	**CONCLUSION AND FUTURE ENHANCEMENTS**	54
	8.1 Conclusion	54
	8.2 Limitations	55
	8.3 Future Enhancements	56
	REFERENCES	57

LIST OF TABLES

Table No.	Title	Page No.
Table 2.1	Literature Coverage of Retail Product Recognition and Shelf Monitoring	7
Table 2.2	Research Gap Identified from the Reviewed Studies	10
Table 5.1	Hardware Requirements	26
Table 5.2	Software Requirements	27
Table 5.3	Dataset Specification	29
Table 5.4	Performance Evaluation Metrics	31
Table 5.5	YOLO11n Phase I Evaluation Results	32
Table 6.1	Functional Test Cases	36
Table 6.2	YOLO11n Model Evaluation Results	38
Table 6.3	Integration Test Cases	39
Table 6.4	Stock Status Test Cases	40
Table 6.5	Alert Test Cases	41
Table 6.6	End-to-End Test	43
Table 6.7	User Acceptance Test	43
Table 7.1	Phase I Implementation Results	45
Table 7.2	YOLO11n Model Evaluation Results	47
Table 7.3	Sample Application Monitoring Results	50
Table 7.4	Overall Phase I Result Summary	51

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 4.1	System Architecture of ShelfAI Phase I	18
Figure 7.1	YOLO11n Object Detection Output	48
Figure 7.2	Visible Object Counting Result	48
Figure 7.3	Shelf Region and Stock Status Result	49
Figure 7.4	ShelfAI Dashboard and Monitoring Interface	49
Figure 7.5	Shelf Monitoring Alert Result	49
Figure 7.6	Historical Monitoring and Scan Results	49

LIST OF ABBREVIATIONS

Table A.1 Abbreviations used in the report.

Abbreviation	Meaning
AI	Artificial Intelligence
YOLO	You Only Look Once
IoT	Internet of Things
POS	Point of Sale
CNN	Convolutional Neural Network
SKU	Stock Keeping Unit
RPC	Retail Product Checkout
mAP	Mean Average Precision
IoU	Intersection over Union
FPS	Frames Per Second
TP	True Positive
FP	False Positive
FN	False Negative
MAE	Mean Absolute Error
SQL	Structured Query Language
ESP32	Espressif 32-bit Microcontroller
OpenCV	Open Source Computer Vision Library
API	Application Programming Interface

The abbreviations listed in Table A.1 are used consistently throughout the report to avoid repeated expansion of commonly used technical terms.

CHAPTER 1

INTRODUCTION

1.1 General

Retail stores contain a large number of products arranged across multiple shelves, and maintaining product availability is an important aspect of effective store management. Products may become unavailable or reach low-stock conditions due to customer purchases, delayed replenishment, incorrect placement, or insufficient monitoring. In conventional retail environments, employees periodically inspect shelves and identify products that are empty or require replenishment. This manual process requires continuous human effort and may become difficult to manage when a large number of shelves have to be monitored regularly. Delayed identification of stock shortages can also affect product availability and customer satisfaction.

Inventory Management and Point-of-Sale (POS) systems are widely used to maintain information about product transactions and inventory quantities. However, transaction records do not always represent the actual physical condition of products on a retail shelf. Products may be misplaced, hidden behind other products, or remain unreplenished after a sale. Therefore, there is a need for a system that can provide visual information about the physical condition of retail shelves in addition to conventional inventory information.

Artificial Intelligence and computer vision provide an approach for automating shelf monitoring. A camera can capture images of retail shelves, and an object detection model can identify visible product instances within the captured image. The detected objects can be counted and analysed to estimate the visible stock level of a shelf. Object detection models in the YOLO family are suitable for this type of application because they can identify object locations and confidence values within an image while providing efficient inference.

However, vision-based monitoring is affected by practical conditions such as product overlap, occlusion, varying lighting, product orientation, and densely arranged items. In addition, a generic object detection model cannot directly identify individual brands or SKU-level products unless it is trained using an appropriate product-specific dataset. These factors need to be considered when developing an automated retail shelf monitoring system.

The proposed project, titled “AI-Powered Retail Shelf Monitoring and Stock Availability Detection,” addresses this problem through a software-based computer vision system. The Phase I implementation uses a trained YOLO11n object detection model to analyse uploaded shelf

images, detect visible product-object instances, count the detected objects, analyse predefined shelf regions, and determine stock conditions using configured minimum-stock thresholds. The resulting information is stored and presented through a database-driven monitoring dashboard. The system is designed to assist store personnel in identifying shelf conditions that require attention and replenishment.

The current Phase I implementation focuses on vision-based shelf monitoring. Physical stock verification using IoT components such as ESP32, HX711, and load cells is considered a Phase II extension. Similarly, individual SKU or brand-level recognition using a project-specific labelled dataset is considered a future enhancement to overcome the limitations of the current generic object-detection model.

1.2 Problem Statement

Retail shelf availability is commonly monitored through manual inspection and transaction-based inventory systems. Manual inspection requires employees to repeatedly check shelves, count products, and identify low-stock or empty conditions. This approach becomes time-consuming and difficult to scale when many shelves need to be monitored.

Transaction-based inventory systems provide useful information about sales and inventory records, but the recorded quantity may not always correspond to the physical condition of products on a shelf. Products can be misplaced, hidden, or left unreplenished, resulting in a difference between recorded inventory and visible shelf availability.

Computer-vision-based monitoring can reduce the dependence on manual inspection by automatically detecting and counting visible objects in shelf images. However, challenges such as occlusion, overlapping products, lighting variations, and generic object classes can affect the accuracy of visual counting. Therefore, a software system is required to automate shelf-image analysis, count visible product instances, evaluate shelf conditions using configurable thresholds, maintain scan records, and provide the results through an accessible monitoring interface.

1.3 Motivation

The primary motivation of the project is to reduce the effort involved in continuous manual monitoring of retail shelves and to provide faster identification of shelf conditions that require attention. An automated vision-based system can analyse shelf images consistently and

provide information about the number of visible objects detected within predefined shelf regions.

Another motivation is to provide a centralized monitoring interface through which store personnel can review current shelf conditions and previous inspection results. By integrating object detection, object counting, threshold-based stock classification, database storage, and dashboard visualization, the system can support a structured approach to retail shelf monitoring.

The limitations of purely visual monitoring also motivate future development of physical stock verification. In Phase II, IoT-based weight sensing can be explored as an additional source of information for verifying physically available stock. Project-specific product recognition can also be introduced through a dedicated labelled dataset and additional model training.

1.4 Objectives

The primary objective of the project is to develop an AI-powered software system for automated monitoring of retail shelf conditions using computer vision and object detection.

The specific objectives of the project are to develop a system that can:

1. Capture or accept retail shelf images as input for automated analysis.
2. Preprocess the input images and perform object detection using a trained YOLO11n model.
3. Detect visible retail object instances and obtain their bounding boxes, confidence scores, and class information.
4. Count the detected object instances within predefined shelf regions.
5. Analyse shelf occupancy and determine the visible stock condition using configured minimum-stock thresholds.
6. Classify the monitored condition as NORMAL, LOW-STOCK, or OUT-OF-STOCK based on the application-level stock thresholds.
7. Store image-analysis results, scan information, shelf details, and stock-status information in a database.
8. Provide a dashboard through which users can monitor shelf conditions and review previous scan results.

9. Generate alerts for configured low-stock or out-of-stock conditions so that responsible staff can take appropriate replenishment action.
10. Evaluate the trained object detection model using standard object-detection metrics such as Precision, Recall, F1-Score, mAP@0.5, and mAP@0.5:0.95.
11. Identify the limitations of generic object detection and provide a foundation for future project-specific SKU recognition and physical stock verification.

1.5 Scope

The scope of the current project is limited to Phase I software-based retail shelf monitoring using camera or uploaded shelf images. The system accepts a shelf image as input, performs image processing and object detection, and analyses the detected objects to determine the visible shelf condition.

The Phase I system uses the SKU110K resized dataset for training and evaluation of the YOLO11n object detection model. The current dataset configuration uses a single generic object class, represented as `object`. Therefore, the model detects visible retail object instances but does not provide individual brand or SKU-level recognition. Product names configured within the application are used for shelf and threshold management and should not be interpreted as classes learned by the current YOLO model.

The system includes image upload, image preprocessing, YOLO11n-based object detection, confidence-based detection filtering, object counting, predefined shelf-region analysis, stock-status classification using configured thresholds, database integration, scan-history management, dashboard visualization, and alert generation.

The current Phase I system does not include physical IoT-based stock measurement. Components such as ESP32, HX711, load cells, and physical weight-based quantity estimation are planned for Phase II. Similarly, project-specific product/SKU recognition, multi-camera monitoring, POS integration, automated purchase ordering, demand forecasting, and automated replenishment are outside the current Phase I scope.

The project is intended to provide a practical foundation for automated retail shelf monitoring. The Phase I implementation demonstrates the feasibility of using computer vision to detect and count visible objects on shelves, while the identified limitations provide directions for subsequent development.

1.6 Significance of the Project

The proposed ShelfAI system is significant because it applies artificial intelligence and computer vision to a practical retail monitoring problem. By automatically analysing shelf images, the system can reduce the amount of repetitive manual inspection required from store personnel and provide structured information about visible shelf conditions.

The use of YOLO11n enables the system to detect multiple visible object instances within a shelf image and obtain their locations and confidence values. The detected objects can then be counted and analysed using application-level shelf configurations and minimum-stock thresholds. This provides a systematic method for identifying conditions that may require replenishment.

The database and dashboard components further improve the usefulness of the system by maintaining scan history and presenting monitoring information through a centralized interface. Store personnel can use the resulting information to identify shelves requiring attention and review previous monitoring activity.

The project also establishes a foundation for future improvements. A project-specific dataset can be developed to enable individual product or SKU recognition, while IoT-based weight sensing can provide an additional physical measurement of stock. The combination of these future capabilities with the current computer-vision system can support more comprehensive retail shelf monitoring and physical stock verification.

Overall, the Phase I project demonstrates how AI-based object detection can be integrated with application-level shelf analysis and database-driven monitoring to support automated retail shelf availability assessment.

CHAPTER 2

LITERATURE SURVEY

2.1 Retail Product Recognition

Franco, Maltoni, and Papi investigated grocery product detection and recognition and highlighted the practical difficulties involved in recognizing products in supermarket environments [1]. Retail shelf images commonly contain multiple products with similar packaging, partial occlusion, different orientations, and variations in illumination. These conditions demonstrate the challenges involved in reliable visual analysis of retail shelves and indicate the need for robust computer-vision techniques rather than simple image-matching approaches.

The findings of this study are relevant to the proposed ShelfAI system because the current system also analyses images containing densely arranged retail objects. The challenges identified in the study provide motivation for using a deep-learning-based object detection approach capable of locating multiple visible objects within a shelf image.

2.2 Dense Shelf Detection and Product Recognition

Goldman et al. studied precise object detection in densely packed scenes [2]. Dense retail shelves present a similar challenge because adjacent products can be closely positioned, partially overlap, or appear with limited visible surface area. Accurate object localization is therefore important when the detected objects are subsequently counted.

The study supports the use of bounding-box-based object detection for retail shelf analysis. In the proposed system, detected object locations are used to determine visible object instances and support subsequent counting and shelf-region analysis.

2.3 Retail Datasets

Wei et al. introduced the Retail Product Checkout dataset, commonly referred to as RPC, as a large-scale benchmark for retail product recognition [3]. The dataset provides a realistic basis for studying product appearance and recognition in retail environments. Bai et al. presented Products-10K, another large-scale product recognition dataset containing diverse retail product images [4]. These datasets demonstrate the importance of collecting sufficiently varied and representative product images when developing models for retail environments.

The reviewed datasets are relevant to the present project because the performance of a computer-vision model depends strongly on the quality, diversity, and annotation of the training data. The current ShelfAI Phase I implementation uses the SKU110K resized dataset

for training and evaluation of the YOLO11n object detection model. However, the current dataset configuration uses a single generic object class, and therefore it does not provide individual brand or SKU-level recognition.

Table 2.1 – Literature Coverage of Retail Product Recognition and Shelf Monitoring

Reference	Main contribution	Relevance to this project
[1]	Grocery product detection and recognition	Establishes retail recognition challenges such as similarity and occlusion.
[2]	Detection in densely packed scenes	Motivates precise localization for closely arranged shelf products.
[3]	RPC large-scale retail dataset	Provides a benchmark for retail product recognition.
[4]	Products-10K large-scale dataset	Demonstrates the need for diverse product appearance data.

2.4 Deep Learning for Retail Product Recognition

Pietrini et al. proposed a deep-learning-based product recognition system for intelligent retail environments [5]. Their work demonstrates how deep learning can be used to connect visual information obtained from retail images with product identities at a fine-grained level. This is relevant to retail shelf monitoring because detecting an object alone is different from identifying the specific product represented by that object.

The study on advances in deep learning-based image recognition of product packaging reviewed the application of deep learning techniques to product-package recognition [6]. The findings reinforce the importance of learned visual features for handling variations in product appearance and complex retail scenes.

These studies indicate that product-specific recognition requires suitable training data and appropriate class definitions. In contrast, the current ShelfAI Phase I model is configured with a single generic object class. Therefore, the present implementation focuses on detecting and counting visible object instances rather than identifying individual brands or SKU-level products. A project-specific product recognition model can be developed as a future enhancement.

2.5 Empty-Shelf and Out-of-Stock Detection

Santra, Ghosh, and Mukherjee proposed a graph-based modelling approach using superpixels for automatic identification of empty shelves in supermarkets [7]. Their work addresses shelf-level availability using visual information and demonstrates that identifying empty shelf regions is a distinct problem from recognizing individual products. This distinction is relevant to ShelfAI because object detection and stock-status analysis are treated as separate stages in the system.

The Sensors study on enhanced out-of-stock detection in retail shelf images applied deep learning to improve the identification of out-of-stock conditions [8]. The study demonstrates the importance of an availability-detection stage in addition to object detection. In the present project, the stock condition is determined after object detection and counting by comparing the detected quantity with configured minimum-stock thresholds.

Allegra et al. investigated out-of-stock detection using egocentric shopping-cart vision [9]. Their work demonstrates that image acquisition method and viewpoint can influence the detection of stock conditions. In comparison, the present project analyses uploaded shelf images using a consistent shelf-oriented monitoring workflow in which predefined shelf regions can be configured for analysis.

De Simone et al. presented an autonomous mobile robot approach for automatic out-of-stock detection in supermarkets [10]. Their work demonstrates that automated shelf inspection can also be implemented using mobile sensing platforms. In contrast, the current ShelfAI Phase I implementation uses a software-based image-analysis workflow rather than a mobile robotic platform.

2.6 Shelf Monitoring and Planogram Compliance

Saqlain et al. proposed a hybrid approach for shelf monitoring and planogram compliance using deep learning and computer vision [11]. Their work is relevant because retail shelf monitoring involves not only determining whether objects are present but also analysing whether products are positioned in their expected shelf locations.

The study supports the importance of spatial shelf information in retail monitoring. In ShelfAI, predefined shelf regions are used to organize detected objects and support region-level analysis. The current application can also apply configured shelf and product-location rules to identify potential placement inconsistencies. Such placement analysis is application-

level logic and should be distinguished from SKU-level recognition performed by a trained AI model.

2.7 Object Detection Models

Ren et al. introduced Faster R-CNN, a region-proposal-based object detection framework [12]. Faster R-CNN established a strong two-stage object detection approach capable of accurate object localization and classification. However, applications requiring frequent image processing may benefit from more computationally efficient one-stage object detection approaches.

Bochkovskiy et al. proposed YOLOv4 with emphasis on the balance between detection speed and accuracy [13]. Wang et al. later presented YOLOv7 and demonstrated further improvements in real-time object detection performance [14]. These developments illustrate the evolution of the YOLO family for efficient object detection.

The reviewed object detection approaches support the selection of a YOLO-family detector for the present project. ShelfAI Phase I uses YOLO11n, a lightweight YOLO model, for detecting visible retail object instances in shelf images. The model generates bounding boxes, confidence scores, and class information, which are subsequently used by the application for object counting and shelf analysis.

2.8 Product Classification and Data Augmentation

Santra, Shaw, and Mukherjee proposed a part-based annotation-free fine-grained classification method for retail products [15]. Fine-grained recognition is important in retail environments where different products may have visually similar packaging. The study demonstrates that detailed visual representation is necessary when the objective is to distinguish between individual products.

The study on generating training images at different angles using Generative Adversarial Networks (GANs) for grocery product image recognition investigated synthetic image generation to improve recognition under viewpoint changes [16]. The work highlights the importance of considering variations in product orientation and appearance during dataset preparation.

These studies are relevant to the future development of ShelfAI because individual SKU recognition would require a sufficiently diverse, product-specific dataset containing

appropriate annotations and variations in product appearance. The current Phase I implementation does not perform SKU-level classification and therefore does not claim brand-specific recognition.

2.9 Research Gap

The reviewed studies demonstrate significant research activity in retail product recognition, dense-scene object detection, empty-shelf analysis, out-of-stock detection, automated shelf inspection, and planogram compliance. Existing approaches address different aspects of retail shelf analysis, including product recognition, object localization, availability detection, and spatial shelf monitoring.

A gap identified from the reviewed literature is the need for an integrated and practical workflow that connects image-based object detection with object counting, predefined shelf-region analysis, configurable stock thresholds, database storage, dashboard monitoring, and alert generation. Many studies concentrate on a particular computer-vision task or experimental dataset rather than providing an end-to-end application-oriented shelf-monitoring workflow.

The present ShelfAI Phase I project addresses this system-level gap by integrating established computer-vision techniques into a single software-based retail shelf monitoring application. The YOLO11n model detects visible object instances, while application-level processing performs counting, shelf-region analysis, threshold-based stock classification, database storage, scan-history management, dashboard presentation, and alerts.

The project does not claim that these individual techniques are novel algorithms. Instead, its contribution lies in implementing and integrating established techniques into a practical retail shelf monitoring prototype and evaluating the performance of the selected object detection model.

The reviewed literature also highlights limitations that remain relevant to the present implementation. In particular, generic object detection does not provide reliable individual SKU or brand recognition without a suitable product-specific dataset. Similarly, vision-based counting may be affected by occlusion, dense product arrangements, lighting conditions, and partially visible objects. These limitations provide the basis for future development involving project-specific product recognition and physical stock verification.

Table 2.2 – Research Gap Identified from the Reviewed Studies

Area	Evidence from literature	Gap addressed by proposed project
Retail product recognition	[1], [3], [4], [5], [6], [15]	Provides a practical shelf-monitoring workflow while recognizing that SKU-level recognition requires a dedicated product-specific dataset.

Dense and complex shelves	[2]	Uses object detection, confidence filtering, and region-based counting to analyse visible shelf objects.
Empty/out-of-stock detection	[7], [8], [9], [10]	Connects object detection and counting with application-level stock-status classification using configurable thresholds.
Shelf monitoring and placement	[11]	Uses predefined shelf regions and application-level configuration for shelf monitoring and potential placement inconsistencies.
Object detection	[12], [13], [14]	Highlights the need for varied training data and provides a basis for future improvement of the project-specific dataset.
Viewpoint and appearance variation	[16]	Include dataset variation and augmentation in model preparation.
Practical monitoring application	[7]–[11]	Integrates detection, counting, shelf analysis, database storage, dashboard monitoring, and alert generation in one application.

Table 2.2 summarizes the major research areas reviewed, the corresponding literature evidence, and the system-level gap addressed by the proposed ShelfAI implementation.

CHAPTER 3

PROPOSED SYSTEM

3.1 System Overview

The proposed system, named ShelfAI, is an AI-based retail shelf monitoring and stock availability detection system designed to assist store staff and managers in monitoring visible shelf conditions. In conventional retail environments, shelf inspection is often performed manually, which can be time-consuming and may lead to delays in identifying low-stock or empty shelf conditions. In addition, inventory information obtained from billing or transaction systems does not always represent the actual physical condition of products displayed on the shelf. ShelfAI addresses this problem by analysing uploaded shelf images using computer vision and applying rule-based stock evaluation.

Phase I of the proposed system is implemented as a software-based solution in which a store staff member or manager uploads an image of a retail shelf through a web application. The uploaded image is transferred to the backend for preprocessing and analysis. The YOLO11n object detection model is then used to identify visible retail object instances in the image. The detected objects are represented using bounding boxes, confidence scores, and the available class information. Since the current model is trained using the single generic class `object`, the system detects visible retail object instances rather than identifying individual brands or SKU types.

After detection, the system filters detections according to the configured confidence threshold and processes the resulting bounding boxes. The detected objects are counted and associated with predefined shelf regions using configured spatial information. The resulting visible object count is compared with the minimum stock threshold configured for the corresponding shelf or product configuration. Based on this comparison, the shelf condition is classified as NORMAL, LOW-STOCK, or OUT-OF-STOCK.

The monitoring results are stored in the application database along with relevant scan information. The web application provides a dashboard through which users can view detection results, object counts, stock status, alerts, and historical monitoring records. Thus, the proposed system provides an integrated workflow from shelf image acquisition to AI-based analysis, stock evaluation, storage, and monitoring.

3.2 Proposed System Workflow

The overall workflow of ShelfAI Phase I begins with the store staff or manager accessing the web application. The user uploads a shelf image through the image upload interface. The backend receives the image and performs the required preprocessing operations before passing it to the YOLO11n object detection model.

The detection model analyses the image and generates bounding boxes, confidence scores, and class information for the detected objects. Detections that do not satisfy the configured confidence threshold are filtered out. The remaining detections are processed to determine their locations and visible counts.

The system then assigns detected objects to predefined shelf regions using configured region coordinates. The number of detected objects within the relevant region is used for stock evaluation. A rule-based evaluation compares the detected count with the configured minimum stock threshold. If no object is detected, the corresponding condition is classified as OUT-OF-STOCK. If the detected count is greater than zero but below the configured minimum threshold, it is classified as LOW-STOCK. If the detected count is equal to or greater than the minimum threshold, the condition is classified as NORMAL.

The resulting stock status and monitoring information are recorded in the database. Alerts are generated when the configured stock rules indicate a condition requiring attention. The results are subsequently made available through the dashboard and historical monitoring interface.

The complete Phase I workflow can therefore be represented as:

Store Staff / Manager → Web Application → Shelf Image Upload → Backend/API → Image Preprocessing → YOLO11n Object Detection → Confidence Filtering → Detection Processing → Visual Object Counting → Shelf Region Assignment → Rule-Based Stock Evaluation → Stock Status → Alert Generation → Database/Storage → Dashboard and Historical Monitoring Results

3.3 Major System Modules

3.3.1 User Interface and Web Application Module

The user interface provides the primary interaction point between the store staff or manager and the ShelfAI system. The web application allows authorized users to access the monitoring functionality, upload shelf images, view analysis results, and monitor historical records.

The dashboard presents the processed information in a form that can be used for shelf-level monitoring and decision-making.

3.3.2 Shelf Image Upload Module

The image upload module allows the user to provide a shelf image for analysis. The uploaded image acts as the primary input to the Phase I system. The module transfers the selected image to the backend, where it is processed before AI-based detection.

Phase I uses uploaded images rather than a continuously operating live camera system. Live camera-based monitoring can be considered as a future enhancement.

3.3.3 Image Preprocessing Module

The preprocessing module prepares the uploaded shelf image for object detection. Operations such as image resizing and normalization are applied as required by the model inference pipeline. OpenCV and the model's supported preprocessing mechanisms are used to prepare the image for YOLO11n analysis.

The purpose of preprocessing is to provide a suitable and consistent input to the detection model while maintaining the visual information required for object detection.

3.3.4 YOLO11n Object Detection Module

The YOLO11n model forms the primary artificial intelligence component of the Phase I system. It analyses the preprocessed shelf image and identifies visible object instances. For each detection, the model provides a bounding box, confidence score, and class information.

The current trained model uses a single generic object class named `object`. Therefore, the current Phase I implementation should be understood as visible retail object detection and counting rather than individual product or SKU recognition. Project-specific SKU recognition is planned as a future enhancement after creating and training an appropriate product-specific dataset.

3.3.5 Confidence Filtering Module

The confidence filtering module removes detections whose confidence scores do not satisfy the configured detection threshold. This reduces the effect of low-confidence predictions on subsequent counting and shelf analysis.

Only the detections that satisfy the configured confidence criterion are passed to the subsequent processing stages.

3.3.6 Detection Processing Module

The detection processing module handles the output generated by YOLO11n after confidence filtering. It processes the detected bounding boxes and their coordinates so that the system can determine the spatial location of each visible object.

The processed coordinates are subsequently used for object counting and shelf-region assignment.

3.3.7 Visual Object Counting Module

The visual object counting module determines the number of detected object instances present in the uploaded shelf image or within a configured shelf region. The count represents the number of visibly detected objects in the image.

The count should not be interpreted as an exact physical inventory quantity because products may be hidden behind other products, partially visible, or affected by image quality and occlusion.

3.3.8 Shelf Region Assignment Module

The shelf region assignment module associates detected objects with predefined shelf regions using configured spatial coordinates. This allows the system to perform analysis for a specific portion of the shelf rather than treating the complete image as a single undivided region.

Shelf and product-related configurations can be maintained at the application level. For example, the system may contain configured shelf information such as A01 Dairy Coolers and A02 Fresh Bakery together with corresponding monitoring thresholds.

These configured names do not represent classes learned by the current YOLO11n model.

3.3.9 Rule-Based Stock Evaluation Module

The rule-based stock evaluation module determines the stock condition using the visible detected count and the configured minimum stock threshold. The evaluation provides a simple and interpretable method for converting object detection results into a shelf monitoring status.

The classification rules used in Phase I are:

- Detected Count = 0 → OUT-OF-STOCK
- 0 < Detected Count < Minimum Threshold → LOW-STOCK
- Detected Count ≥ Minimum Threshold → NORMAL

The thresholds are application configurations and are used to interpret the detected visible count for monitoring purposes.

3.3.10 Stock Status Module

The stock status module presents the result of the rule-based evaluation as one of three conditions: NORMAL, LOW-STOCK, or OUT-OF-STOCK. This provides store staff and managers with an immediate indication of the observed shelf condition.

The status is based on the visible objects detected in the uploaded image and the configured threshold. Therefore, it represents an image-based shelf availability assessment rather than a complete physical inventory measurement.

3.3.11 Alert Generation Module

The alert generation module produces monitoring alerts when the rule-based evaluation identifies a condition requiring attention. Low-stock and out-of-stock conditions can therefore be recorded and presented as alerts for monitoring purposes.

The alert information is stored together with the corresponding monitoring record so that the user can review the condition through the application.

3.3.12 Database and Storage Module

The database and storage layer maintains the information generated during the monitoring process. Monitoring records, shelf information, detection results, stock status, and alert information can be stored for later retrieval.

Maintaining historical records allows the application to provide a record of previous shelf scans and their corresponding monitoring results.

3.3.13 Dashboard and Historical Monitoring Module

The dashboard provides a centralized interface for viewing the results generated by the ShelfAI system. The monitoring interface can display visual detection results, detected counts, stock conditions, alerts, and historical monitoring information.

The dashboard converts the output of the AI and rule-based processing stages into information that can be interpreted by store staff and managers.

3.4 Proposed System Advantages

The proposed ShelfAI system provides several practical advantages for image-based retail shelf monitoring. It reduces the dependency on completely manual shelf inspection by automating the detection and counting of visible shelf objects. The use of YOLO11n provides an efficient object detection mechanism suitable for analysing uploaded shelf images.

The system also combines AI-based detection with rule-based stock evaluation. Instead of presenting only bounding boxes, ShelfAI converts the detected visible object count into an interpretable shelf condition using configured minimum thresholds. The database and dashboard further provide centralized access to monitoring results and historical records.

Another advantage is the modular design of the system. The current Phase I software architecture can be extended in future development by introducing project-specific product datasets, SKU-level recognition, additional image sources, and physical stock measurement without changing the basic concept of storing and presenting monitoring results.

CHAPTER 4

PROPOSED METHODOLOGY AND SYSTEM DESIGN

4.1 Introduction

The ShelfAI Phase I system is designed as a software-based image analysis pipeline for monitoring the visible condition of retail shelves. The methodology combines artificial intelligence-based object detection with image processing, spatial shelf-region analysis, object counting, and rule-based stock evaluation. The processed results are stored in a database and presented through a web-based dashboard.

The Phase I architecture consists of a user interface layer, AI and computer vision layer, backend and application logic layer, output and observation layer, and data storage layer. The system begins with a shelf image uploaded by the store staff or manager and ends with a stock status, monitoring record, and corresponding dashboard information.

The complete methodology is designed to maintain a clear separation between AI-based detection and application-level decision rules. YOLO11n performs object detection, while counting, shelf-region assignment, threshold comparison, status generation, and alert generation are handled by the application logic.

4.2 Phase I System Architecture

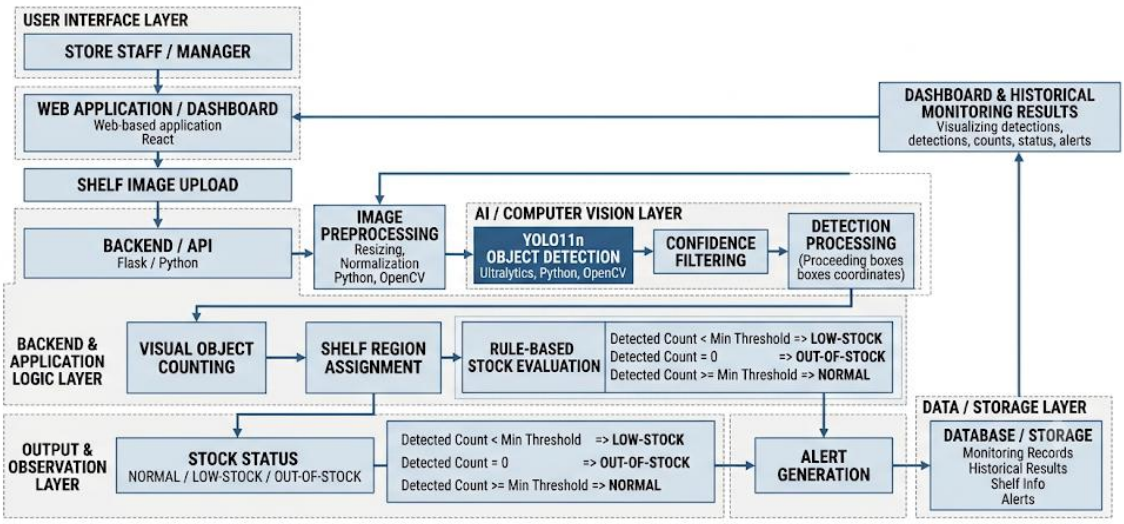


Figure 4.1 – System Architecture of ShelfAI Phase I

Figure 4.1 shows the Phase I system architecture of ShelfAI, illustrating the processing flow from uploaded shelf image input through image preprocessing, YOLO11n object detection, confidence filtering, visible object counting, shelf-region analysis, rule-based stock-status evaluation, database storage, and dashboard and alert presentation. The architecture is organized into the following functional layers.

The User Interface Layer contains the store staff or manager interface and the web application/dashboard. It provides the interaction point through which users upload shelf images and view monitoring results.

The AI and Computer Vision Layer contains image preprocessing, YOLO11n object detection, confidence filtering, and detection processing. This layer is responsible for analysing the uploaded image and producing reliable detection information for subsequent processing.

The Backend and Application Logic Layer contains visual object counting, shelf-region assignment, and rule-based stock evaluation. These operations convert the AI detection output into meaningful shelf-level monitoring information.

The Output and Observation Layer presents the resulting stock status and monitoring condition to the user. The status may be NORMAL, LOW-STOCK, or OUT-OF-STOCK depending on the detected visible count and configured threshold.

The Data and Storage Layer maintains monitoring records, historical results, shelf information, and alert information for subsequent retrieval and dashboard presentation.

The overall Phase I processing sequence is:

Shelf Image → Image Preprocessing → YOLO11n Object Detection → Confidence Filtering → Detection Processing → Visual Object Counting → Shelf Region Assignment → Rule-Based Stock Evaluation → Stock Status → Alert Generation → Database/Storage → Dashboard and Historical Monitoring Results

4.3 Image Acquisition and Input

The primary input to Phase I is a digital image of a retail shelf. The image is supplied by the user through the web application's shelf image upload interface.

The uploaded image is transferred to the backend for processing. The image contains the visible shelf area that is intended for analysis. Since the current Phase I implementation is based on uploaded images, it does not depend on continuous live camera streaming.

Future versions may support direct camera input or periodic camera-based image acquisition. Such functionality is outside the current Phase I implementation.

4.4 Image Preprocessing

Before object detection, the uploaded image is prepared for analysis. Image preprocessing ensures that the input satisfies the requirements of the object detection pipeline.

The preprocessing stage may include image resizing, normalization, and image-format handling. OpenCV-based processing can be used where required by the application, while the YOLO inference pipeline performs its supported input preparation.

The objective of preprocessing is to provide a consistent image representation to the detection model while retaining the visual information required to identify visible objects.

4.5 YOLO11n Object Detection

YOLO11n is the primary artificial intelligence algorithm used in ShelfAI Phase I. YOLO11n is a lightweight object detection model from the Ultralytics YOLO family and is used to identify visible object instances in the uploaded shelf image.

For each detected object, the model produces information including the bounding box coordinates, confidence score, and predicted class.

The detection output can be represented as:

- represents the bounding box of the detected object,
- represents the predicted class,
- represents the confidence score, and
- represents the total number of detections produced by the model.

The current trained model uses a single generic class:

Therefore, the model currently identifies visible retail object instances rather than individual product names, brands, or SKUs.

Individual product/SKU recognition requires a project-specific labelled dataset and a corresponding product-specific training process, which is considered a future enhancement.

4.6 Confidence Filtering

The raw detection output may contain predictions with different confidence levels. To reduce the effect of uncertain predictions, the system applies a configured confidence threshold.

A detection is retained when:

where is the confidence score of the detection and is the configured confidence threshold.

Detections that do not satisfy this condition are excluded from subsequent counting and shelf analysis.

Confidence filtering therefore acts as an intermediate quality-control stage between YOLO11n detection and the application-level processing stages.

4.7 Detection Processing

After confidence filtering, the retained bounding boxes and their coordinates are processed by the application. The purpose of this stage is to prepare the detection information for spatial analysis and counting.

Each retained detection contains its location within the image. These coordinates are used to determine whether the detected object belongs to a predefined shelf region.

The detection processing stage does not independently identify a product by brand or SKU. It operates on the object detection information generated by YOLO11n.

4.8 Visual Object Counting

The visual object counting stage determines the number of retained detections in the relevant image or shelf region.

If represents a predefined shelf region, the visible object count within that region can be represented as:

where:

- is the visible detected object count in region ,
- is the bounding box of a detected object,
- is an indicator function that returns 1 when the detection belongs to the region and 0 otherwise.

The resulting count represents the number of visibly detected object instances.

It does not necessarily represent the actual physical quantity of products present on the shelf because products can be occluded, partially visible, stacked behind other products, or missed because of image quality and detection limitations.

4.9 Shelf Region Assignment

The shelf region assignment stage associates detected objects with predefined regions of the shelf. These regions are maintained using configured spatial coordinates or application-level shelf information.

For example, the application may contain shelf configurations such as A01 Dairy Coolers and A02 Fresh Bakery. Each configured shelf can have associated monitoring information and minimum stock thresholds.

The region assignment operation can be represented as:

where represents one configured shelf region.

A detected object is associated with a region according to the spatial relationship between its bounding box and the configured region coordinates.

This approach enables the system to perform localized shelf monitoring instead of treating the entire uploaded image as one undivided area.

4.10 Rule-Based Stock Evaluation

After the visible object count is determined for the relevant shelf region, the system compares the count with the configured minimum stock threshold.

Let:

- = detected visible object count,
- = configured minimum stock threshold.

The stock condition is determined using the following rules:

This rule-based approach provides an interpretable method for converting the visual detection result into a shelf monitoring condition.

The threshold is an application-level configuration. It is not learned automatically by the YOLO11n model.

4.11 Stock Status Generation

The result of the rule-based evaluation is presented as the stock status of the monitored shelf region.

When no visible object is detected, the system assigns the OUT-OF-STOCK condition. When visible objects are detected but their count is below the configured minimum threshold, the system assigns LOW-STOCK. When the detected count meets or exceeds the threshold, the system assigns NORMAL.

The generated status is based on image-visible objects and should therefore be interpreted as an image-based shelf availability condition rather than an exact physical inventory value.

4.12 Alert Generation

The alert generation stage uses the stock evaluation result to identify monitoring conditions that require attention.

A low-stock or out-of-stock condition can generate an alert for the corresponding shelf monitoring record. The alert information is stored with the monitoring result and can subsequently be displayed through the dashboard.

The alert mechanism allows users to identify shelf conditions that may require physical inspection or replenishment.

4.13 Database and Storage

The database layer stores information generated during shelf monitoring. The stored information can include shelf details, scan information, detection results, object counts, stock status, and alert records.

The storage of monitoring results provides persistence across multiple shelf scans and enables the application to maintain historical monitoring information.

The database therefore acts as the connection between the processing pipeline and the historical monitoring interface.

4.14 Dashboard and Historical Result Presentation

The dashboard provides the final observation interface for the processed shelf monitoring results.

The results generated by the AI and application logic are presented through the web application. Depending on the available application functionality, the dashboard can display detected objects, visible object counts, stock status, alerts, and previous monitoring records.

Historical monitoring allows users to review previous scans rather than relying only on the latest shelf analysis.

The dashboard therefore serves as the final presentation layer of the Phase I architecture.

4.15 Algorithmic Workflow of Phase I

The complete algorithmic workflow of ShelfAI Phase I can be summarized as follows:

Step 1: The store staff or manager accesses the ShelfAI web application.

Step 2: The user uploads a retail shelf image.

Step 3: The backend receives the uploaded image.

Step 4: The image is preprocessed for model analysis.

Step 5: YOLO11n performs object detection and generates bounding boxes, confidence scores, and class information.

Step 6: Detections below the configured confidence threshold are filtered out.

Step 7: The retained bounding boxes and coordinates are processed.

Step 8: Visible object instances are counted and assigned to the relevant predefined shelf regions.

Step 9: The detected visible count is compared with the configured minimum stock threshold.

Step 10: The shelf condition is classified as NORMAL, LOW-STOCK, or OUT-OF-STOCK.

Step 11: Alerts and monitoring information are generated, stored in the database, and presented through the dashboard and historical monitoring interface.

4.16 Algorithms Used in the Proposed System

The main computational components used in the Phase I system are YOLO11n object detection, image preprocessing, confidence-based filtering, spatial region assignment, object counting, and rule-based threshold evaluation.

YOLO11n is responsible for the AI-based detection of visible object instances. Image preprocessing prepares the input image for inference. Confidence filtering removes low-confidence detections. Spatial region assignment determines the relevant shelf region for each detection. Object counting determines the visible number of retained detections. Finally, rule-based threshold evaluation converts the detected count into a stock status.

The system should not be described as implementing CNN as a separate independent algorithm. Convolutional neural network operations are part of the underlying deep learning architecture of the object detection model. The primary named AI model used by ShelfAI Phase I is YOLO11n.

4.17 Phase I Limitations and Phase II Extension

The Phase I methodology is based on image-visible object detection and therefore has limitations associated with occlusion, overlapping products, image quality, lighting conditions, viewing angle, and objects that are not visually detectable.

The current YOLO11n model also uses a single generic object class. Consequently, the Phase I system cannot reliably distinguish individual brands or SKU categories solely from the current detection model.

The future Phase II extension is intended to improve physical stock verification by incorporating IoT-based weight measurement using components such as ESP32, HX711, and load cells. A project-specific dataset can also be created and used to train a product-specific model for individual SKU recognition.

These components are future extensions and are not part of the implemented Phase I architecture.

CHAPTER 5

SYSTEM REQUIREMENTS AND DATASET SPECIFICATION

5.1 Introduction

This chapter presents the hardware requirements, software requirements, development environment, dataset specification, database requirements, and performance evaluation metrics of the ShelfAI Phase I system. The requirements described in this chapter correspond to the implemented software-based retail shelf monitoring system.

Phase I is designed to analyse uploaded retail shelf images using the YOLO11n object detection model. The system does not require physical IoT sensors, load cells, ESP32, or a live camera for its current operation. These components are considered future extensions under Phase II. The Phase I implementation consists of a computer-based development and inference environment, image processing libraries, the YOLO11n model, a web application, database storage, and a dashboard for presenting monitoring results.

5.2 Hardware Requirements

The hardware requirements for Phase I are minimal because the system operates on uploaded shelf images and performs software-based image analysis.

Table 5.1 – Hardware Requirements

Component	Specification / Use
Computer or Laptop	Used for application development, YOLO11n model training and inference, database operation, and dashboard execution.
Processor	Intel Core i5 or equivalent processor suitable for Python-based development and CPU inference.
RAM	Minimum 8 GB RAM recommended for development, application execution, and model inference.
Storage	Sufficient local storage for the project source code, dataset, trained model, application files, and generated results.

Table 5.1 lists the hardware resources used for the development and evaluation of the ShelfAI Phase I system.

The Phase I system can operate on a standard computer or laptop without requiring a dedicated GPU. During development and evaluation, the YOLO11n model was executed

using CPU-based inference. The system therefore provides a practical software-based implementation suitable for environments where dedicated GPU hardware is not available.

A camera is not a required hardware component for Phase I because the current system receives shelf images through the web application's upload interface. Similarly, ESP32, load cells, HX711, and other IoT components are not required for the current Phase I implementation.

5.3 Software Requirements

The ShelfAI Phase I system uses Python-based AI and backend technologies together with a web-based user interface and database.

Table 5.2 – Software Requirements

Software / Technology	Specific Use in the Project
Python	Primary programming language used for AI model integration, image processing, backend logic, and application development.
Ultralytics YOLO	Framework used to train, validate, and perform inference using the YOLO11n object detection model.
YOLO11n	Primary object detection model used to detect visible retail object instances in shelf images.
PyTorch	Deep learning framework used as part of the YOLO training and inference environment.
OpenCV	Used for image processing and handling of image-related operations.
NumPy	Used for numerical operations and processing of detection-related data.
FAST API	Backend/API framework used to provide application services and connect the AI processing pipeline with the web application.
React	Frontend technology used to implement the web-based user interface and dashboard.
MySQL	Database system used for storing application and shelf monitoring information.

Software / Technology	Specific Use in the Project
Git	Version control system used for source-code management.
Visual Studio Code / Development Environment	Used for project development, debugging, configuration, and application execution.

Table 5.2 lists the software technologies and their respective roles in the ShelfAI Phase I implementation.

The Phase I software environment integrates the AI model with the backend application and web interface. Python and the associated computer vision and deep learning libraries are used for model processing, while Flask provides backend services. React provides the user-facing web interface, and MySQL provides persistent storage for application and monitoring information.

The current implementation does not require an ESP32 programming environment or sensor-specific software because IoT-based physical stock measurement is not part of Phase I.

5.4 Development Environment

The ShelfAI Phase I system was developed and evaluated in a Windows-based computer environment. The development environment includes Python, the Ultralytics framework, PyTorch, OpenCV, NumPy, Flask, React, and MySQL.

The YOLO model training and validation were performed using CPU-based computation. The trained YOLO11n model contains approximately 2.58 million parameters and requires approximately 6.3 GFLOPs according to the model summary.

The development environment is organized so that the AI training and inference components are separated from the web application and database components. This allows the trained model to be integrated into the backend without requiring the entire training environment during normal application operation.

5.5 Dataset Specification

The dataset used for the Phase I object detection model is the SKU110K resized 640 retail shelf dataset obtained from Kaggle. The dataset contains dense retail shelf images in which multiple visible objects appear within the same shelf scene.

The dataset is used to train the YOLO11n object detection model for detecting visible retail object instances. The current Phase I dataset configuration contains a single detection class named object.

Therefore, the current model does not learn individual product names, brands, or SKU categories. For example, application configurations may contain names such as Fresh Milk 1L, Greek Yogurt 500g, or Artisan Sliced Bread, but these names are not individual classes learned by the current YOLO11n model.

Table 5.3 – Dataset Specification

Dataset Feature	Phase I Specification
Dataset	SKU110K resized 640
Dataset Type	Retail shelf object detection dataset
Task	Object detection
Annotation Type	Bounding-box annotations
Number of Classes	1
Class Name	object
Training Images	8,185
Validation Images	584
Testing Images	2,920
Image Format	Retail shelf images suitable for YOLO-based detection
Annotation Format	YOLO-compatible bounding-box annotations
Model	YOLO11n
Training Framework	Ultralytics
Main Purpose	Detection of visible retail object instances

Table 5.3 summarizes the dataset, annotation, class, model, and training configuration used for the ShelfAI Phase I object detection system.

The dataset is divided into training, validation, and testing subsets. The training subset is used

for model learning, while the validation subset is used to evaluate the trained model during development. The test subset is maintained for independent evaluation where required.

The validation set used for the reported Phase I evaluation contains 584 images and 89,934 annotated object instances.

5.6 Dataset Characteristics

Retail shelf images contain several challenges that affect object detection performance. Products can occur close to one another, overlap with adjacent products, appear at different orientations, and be partially hidden by other products. Variations in lighting, shelf arrangement, image quality, and object scale can also affect detection.

The SKU110K dataset provides dense retail shelf scenes that are useful for developing an initial object detection model for the ShelfAI Phase I system. However, because the current dataset and model use a generic object class, the model is not sufficient for reliable individual SKU-level recognition.

For future product-specific recognition, a dedicated dataset containing the target products and corresponding product-level class annotations should be collected and trained. Such a dataset should represent the actual shelf arrangement, product packaging, lighting conditions, viewing angles, and occlusion conditions expected in the intended retail environment.

5.7 Dataset Preparation and Annotation

The dataset used for YOLO11n training contains bounding-box annotations corresponding to visible object instances. Each annotated object is represented by its location within the image.

For YOLO-based object detection, the bounding box is represented using normalized coordinates corresponding to the object center and its width and height. The annotation information allows the model to learn the relationship between the visual appearance of retail objects and their spatial locations.

Since the current dataset configuration contains only one class, all annotated target instances belong to the generic object category. This configuration supports visible object detection and counting but does not provide the class-level information required for individual product identification.

5.8 Model Training Specification

The YOLO11n model was selected as the primary object detection model for Phase I because it provides a lightweight architecture suitable for object detection while requiring relatively limited computational resources compared with larger detection models.

The training process uses the SKU110K resized 640 dataset and its YOLO-compatible configuration. During training, the model learns to localize visible object instances using the available bounding-box annotations.

The trained model is subsequently validated using the validation dataset. The model used for Phase I evaluation is the trained YOLO11n model generated during the project development process.

The model summary contains approximately 2,582,347 parameters and approximately 6.3 GFLOPs.

5.9 Database Requirements

The database provides persistent storage for the information generated by the ShelfAI application. It maintains application and monitoring information required by the dashboard and historical monitoring functions.

The database can contain information related to users, shelf configurations, uploaded scans, detection results, object counts, stock status, alerts, and historical monitoring records.

The current Phase I database is focused on software-based monitoring information. Physical sensor readings and weight-derived stock quantities are not stored as part of the implemented Phase I workflow because IoT-based stock measurement belongs to Phase II.

The application should access database resources through controlled backend interfaces, and sensitive database credentials should be maintained through appropriate configuration mechanisms rather than being hard-coded directly into source files.

5.10 Performance Evaluation Metrics

The performance of the Phase I object detection model is evaluated using standard object detection metrics. The evaluation focuses on the ability of YOLO11n to detect visible retail object instances in shelf images.

Table 5.4 – Performance Evaluation Metrics

Metric	Purpose
Precision	Measures the proportion of predicted detections that are correct.
Recall	Measures the proportion of ground-truth objects that are successfully detected.
F1-score	Provides a combined measure of precision and recall.
mAP@0.5	Measures mean average precision at an Intersection over Union threshold of 0.5.
mAP@0.5:0.95	Measures mean average precision averaged across IoU thresholds from 0.5 to 0.95.
Inference Time	Measures the time required for model inference on an input image under the evaluation environment.

Table 5.4 defines the performance metrics used to evaluate the YOLO11n object detection model.
Intersection over Union (IoU) is used in object detection evaluation to measure the overlap between predicted and ground-truth bounding boxes. The mAP metrics incorporate IoU-based evaluation when determining detection performance.

The quantitative evaluation results obtained from the trained YOLO11n model are presented and discussed in Chapter 7.

5.11 Application-Level Stock Evaluation

The object detection metrics described above evaluate the performance of the AI detection model. However, ShelfAI also performs application-level stock evaluation after detection.

The detected visible object count is compared with a configured minimum stock threshold. The resulting condition is classified as NORMAL, LOW-STOCK, or OUT-OF-STOCK.

The evaluation logic is:

where represents the visible detected object count and represents the configured minimum threshold.

This application-level evaluation is separate from the YOLO model's object detection metrics. A correct stock-status decision depends not only on object detection but also on the detected count, shelf-region assignment, and configured threshold.

CHAPTER 6

TESTING

6.1 Introduction

Testing is an essential stage in the development of the ShelfAI system to verify that the implemented modules operate according to the specified requirements. The testing process for Phase I focuses on the complete software-based workflow, beginning with user authentication and shelf image upload and continuing through image processing, YOLO11n object detection, confidence filtering, object counting, shelf-region analysis, stock-status evaluation, alert generation, database storage, and dashboard presentation.

The testing was performed using the implemented ShelfAI application and the trained YOLO11n model. Functional testing was used to verify individual system operations, while integration testing was used to verify the interaction between the major application modules. Model evaluation was performed separately using the validation dataset and standard object detection metrics.

The Phase I testing confirms the operation of the implemented software workflow. IoT-based sensing using ESP32, load cells, and HX711 is outside the scope of Phase I and is therefore not included as a tested component of the current system.

6.2 Testing Strategy

The testing strategy was designed around the actual Phase I processing workflow of ShelfAI. Each major module was tested using appropriate inputs and its output was compared with the expected behaviour.

The functional tests covered authentication, image upload, image preprocessing, object detection, confidence filtering, detection processing, visible object counting, shelf-region assignment, rule-based stock evaluation, alert generation, database storage, dashboard display, and historical monitoring.

The AI model was evaluated using the validation dataset to measure its detection performance. Precision, recall, F1-score, mAP@0.5, mAP@0.5:0.95, and inference time were considered for model evaluation.

All implemented Phase I functional and integration test cases were successfully completed and passed.

6.3 Functional Test Cases

Table 6.1 – Functional Test Cases

Test Case	Test Description	Input / Condition	Expected Result	Actual Result	Status
TC01	Valid login	Valid username and password	User is authenticated and directed to the main interface.	User successfully logged in and accessed the application.	PASS
TC02	Invalid login	Incorrect username or password	Authentication is rejected and an error is displayed.	Invalid credentials were rejected and the authentication error was displayed.	PASS
TC03	Empty login fields	Username or password left empty	System prevents invalid submission and displays appropriate validation.	Empty-field validation was handled by the application.	PASS
TC04	Shelf image upload	Valid retail shelf image	Image is accepted and transferred for processing.	Shelf image was successfully uploaded and processed.	PASS
TC05	Image preprocessing	Uploaded shelf image	Image is prepared for YOLO11n inference.	Image was successfully prepared for detection.	PASS
TC06	Object detection	Valid shelf image	YOLO11n generates bounding boxes, confidence scores, and class information for detected objects.	Visible retail object instances were detected and bounding boxes were generated.	PASS
TC07	Confidence filtering	Detection results with different confidence scores	Detections below the configured confidence threshold are filtered and valid detections are retained.	Confidence filtering was successfully applied to the detection results.	PASS

Test Case	Test Description	Input / Condition	Expected Result	Actual Result	Status
TC08	Detection processing	Filtered bounding boxes	Detection coordinates are processed for subsequent shelf analysis.	Detected bounding-box information was successfully processed.	PASS
TC09	Shelf-region assignment	Detected objects within configured shelf regions	Valid detections are associated with their corresponding configured shelf regions.	Detections were successfully processed according to the configured shelf regions.	PASS
TC10	Visual object counting	Shelf image containing visible objects	System calculates the number of detected visible object instances.	Visible object counts were successfully generated.	PASS
TC11	Stock classification	Detected count and configured minimum threshold	System generates NORMAL, LOW-STOCK, or OUT-OF-STOCK status according to the configured rules.	Stock status was successfully generated according to the configured threshold.	PASS
TC12	Alert generation	Low-stock or out-of-stock condition	Appropriate monitoring alert is generated.	Alerts were successfully generated for applicable monitoring conditions.	PASS
TC13	Database update	Completed shelf monitoring scan	Monitoring information is stored in the database.	Monitoring results were successfully stored.	PASS
TC14	Dashboard display	Completed monitoring record	Detection results, counts, status, and relevant monitoring information are displayed.	Monitoring information was successfully displayed on the dashboard.	PASS
TC15	Scan history	Previously	Historical monitoring	Previous scan	PASS

Test Case	Test Description	Input / Condition	Expected Result	Actual Result	Status
		completed monitoring scan	record can be retrieved and displayed.	information was successfully available through the monitoring interface.	

Table 6.1 presents the functional test cases used to verify the major Phase I application functions.

6.4 AI Model Testing and Evaluation

The YOLO11n model was evaluated using the validation dataset to determine its ability to detect visible retail object instances. The evaluation was performed against the annotated validation data using standard object detection metrics.

Table 6.2 – YOLO11n Model Evaluation Results

Metric	Result
Precision	76.4%
Recall	68.1%
F1-score	71.9%
mAP@0.5	76.1%
mAP@0.5:0.95	43.7%
Validation Images	584
Validation Instances	89,934
Observed Inference Time	81.1 ms/image

Table 6.2 presents the quantitative evaluation results obtained from the trained YOLO11n model on the validation dataset.

The model achieved a precision of 76.4%, indicating that a substantial proportion of the generated detections corresponded to annotated objects. The recall of 68.1% indicates that the

model detected a considerable proportion of the ground-truth object instances in the validation set.

The F1-score of 71.9% provides a combined measure of precision and recall. The model achieved an mAP@0.5 of 76.1%, while the stricter mAP@0.5:0.95 metric was 43.7%. The difference between these values reflects the greater difficulty of achieving accurate bounding-box localization under stricter Intersection over Union criteria.

The observed model inference time was 81.1 ms per image under the CPU-based evaluation environment. This value represents model inference time and does not represent the complete end-to-end response time of the web application.

Since the current YOLO11n model uses a single generic object class, these results represent generic visible object detection performance and should not be interpreted as individual product or SKU recognition accuracy.

6.5 Integration Testing

Integration testing was performed to verify the interaction between the major modules of the ShelfAI Phase I system. The purpose was to ensure that the output of one processing stage was correctly transferred to the next stage.

Table 6.3 – Integration Test Cases

Integration Test	Components Combined	Expected Behaviour	Actual Result	Status
IT01	Web Application + Image Upload + Backend	Uploaded shelf image is transferred successfully from the user interface to the backend.	Image was successfully transferred to the backend for processing.	PASS
IT02	Backend + Image Preprocessing + YOLO11n	Uploaded image is prepared and passed to the detection model.	Image was successfully processed and passed to YOLO11n.	PASS
IT03	YOLO11n + Confidence Filtering + Detection Processing	Detection results are filtered and processed correctly.	Detection results were successfully filtered and processed.	PASS

Integration Test	Components Combined	Expected Behaviour	Actual Result	Status
IT04	Detection Processing + Object Counting	Processed detections are converted into visible object counts.	Visible object counts were successfully generated.	PASS
IT05	Object Counting + Shelf Region Assignment	Detected objects are associated with the configured shelf regions.	Region-wise detection processing was successfully performed.	PASS
IT06	Shelf Region Assignment + Stock Evaluation	Detected count is compared with the configured minimum threshold.	Appropriate stock status was successfully generated.	PASS
IT07	Stock Evaluation + Alert Generation	Low-stock or out-of-stock conditions generate the corresponding alert.	Applicable alerts were successfully generated.	PASS
IT08	Monitoring Result + Database	Monitoring results are stored for future retrieval.	Monitoring records were successfully stored.	PASS
IT09	Database + Dashboard	Stored monitoring information is retrieved and displayed.	Monitoring information was successfully displayed on the dashboard.	PASS
IT10	Complete Phase I Workflow	Image upload proceeds through AI analysis, counting, stock evaluation, storage, and result presentation.	Complete Phase I workflow operated successfully.	PASS

Table 6.3 presents the integration tests performed between the major ShelfAI processing modules.

6.6 Stock Status and Rule-Based Testing

The stock evaluation component was tested using the configured minimum-stock threshold rules. The purpose of this testing was to verify that the detected visible count was converted into the appropriate shelf condition.

Table 6.4 – Stock Status Test Cases

Test Case	Detected Visible Count	Minimum Threshold	Expected Status	Actual Status	Status
ST01	0	Any positive threshold	OUT-OF-STOCK	OUT-OF-STOCK	PASS
ST02	Below minimum threshold	Configured threshold	LOW-STOCK	LOW-STOCK	PASS
ST03	Equal to minimum threshold	Configured threshold	NORMAL	NORMAL	PASS
ST04	Greater than minimum threshold	Configured threshold	NORMAL	NORMAL	PASS

Table 6.4 presents the test cases used to verify the rule-based stock-status classification. The results confirm that the rule-based stock evaluation operates according to the defined decision logic. A detected count of zero produces an OUT-OF-STOCK condition. A positive count below the configured minimum threshold produces a LOW-STOCK condition, while a count equal to or greater than the threshold produces a NORMAL condition.

The stock status represents an image-based assessment derived from visible detected objects and should not be interpreted as an exact physical inventory quantity.

6.7 Alert Testing

Alert testing was performed to verify that the system generates monitoring alerts when the evaluated shelf condition requires attention.

Table 6.5 – Alert Test Cases

Test Case	Condition	Expected Result	Actual Result	Status
AT01	Normal stock condition	No low-stock or out-of-stock alert is generated.	Normal condition was processed without an applicable stock shortage alert.	PASS
AT02	Low-stock condition	Low-stock monitoring alert is generated.	Low-stock condition was successfully identified and alert information was generated.	PASS
AT03	Out-of-stock condition	Out-of-stock monitoring alert is generated.	Out-of-stock condition was successfully identified and alert information was generated.	PASS

Table 6.5 presents the test cases used to verify alert generation for different stock conditions. The alert tests confirm that the application-level stock status is correctly connected to the monitoring alert mechanism.

6.8 Database and Dashboard Testing

Database and dashboard testing was performed to verify that monitoring results generated by the processing pipeline are stored and subsequently presented to the user.

The database testing confirmed that monitoring information generated after a shelf scan could be stored as application records. Dashboard testing confirmed that the stored monitoring information could be accessed and displayed through the monitoring interface.

The tested workflow was:

Shelf Image → AI Detection → Object Count → Stock Evaluation → Monitoring Record → Database → Dashboard

The successful completion of this workflow confirms the integration between the AI processing, application logic, database, and user interface layers.

6.9 End-to-End Testing

End-to-end testing was performed to verify the complete Phase I workflow from user input to final monitoring output.

The test begins when the store staff member or manager uploads a shelf image through the web application. The image is transferred to the backend and prepared for AI analysis.

YOLO11n detects visible object instances and generates the corresponding detection information. Confidence filtering and detection processing are then applied.

The retained detections are counted and associated with the configured shelf region. The detected visible count is compared with the configured minimum threshold, and the appropriate stock status is generated. Applicable alerts are generated, and the monitoring information is stored in the database. Finally, the result is made available through the dashboard and historical monitoring interface.

The complete Phase I workflow was successfully executed, and the corresponding test case passed.

Table 6.6 – End-to-End Test

Test Case	Input	Expected Result	Actual Result	Status
E2E01	Valid shelf image uploaded through the ShelfAI application	Image is processed, objects are detected and counted, shelf status is generated, results are stored, and monitoring information is displayed.	Complete Phase I processing workflow executed successfully and monitoring results were generated and displayed.	PASS

Table 6.6 presents the end-to-end test of the complete Phase I monitoring workflow.

6.10 User Acceptance Testing

User acceptance testing was performed to verify the usability of the implemented monitoring interface from the perspective of the intended application user. The testing focused on the ability of a user to access the application, upload a shelf image, initiate monitoring, understand the generated detection and stock information, and review monitoring results.

The implemented interface successfully supported the required Phase I monitoring workflow. Users could interact with the web application, submit shelf images, and view the resulting monitoring information through the dashboard.

The user acceptance test for the implemented Phase I functionality was therefore considered successful.

Table 6.7 – User Acceptance Test

Test Case	User Action	Expected Result	Actual Result	Status
UAT01	Login, upload shelf image, execute monitoring, and view result	User can complete the monitoring workflow and understand the displayed result.	User successfully completed the workflow and accessed the monitoring result.	PASS

Table 6.7 presents the user acceptance test performed on the implemented Phase I workflow.

6.11 Testing Summary

The testing activities conducted for ShelfAI Phase I covered the major implemented software components of the system. Functional testing verified authentication, image upload, preprocessing, object detection, confidence filtering, detection processing, counting, shelf-region assignment, stock evaluation, alerts, database storage, dashboard presentation, and scan history.

Integration testing confirmed that the outputs of the individual modules were correctly transferred between successive stages of the processing pipeline. End-to-end testing further confirmed the operation of the complete Phase I workflow from shelf image upload to final monitoring result.

The YOLO11n model was independently evaluated using the validation dataset and achieved a precision of 76.4%, recall of 68.1%, F1-score of 71.9%, mAP@0.5 of 76.1%, and mAP@0.5:0.95 of 43.7%, with an observed inference time of 81.1 ms per image under the CPU-based evaluation environment.

All reported Phase I functional, integration, stock-rule, alert, end-to-end, and user acceptance test cases were successfully completed and marked PASS. IoT-based physical stock measurement and multimodal visual-weight verification are excluded from the Phase I testing scope because they belong to the planned Phase II extension.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Introduction

This chapter presents the results obtained from the implementation and evaluation of the ShelfAI Phase I system. Phase I implements a software-based retail shelf monitoring workflow in which an uploaded shelf image is processed using the YOLO11n object detection model. The detected visible objects are subsequently counted, associated with configured shelf regions, evaluated against minimum-stock thresholds, stored in the database, and presented through the monitoring dashboard.

The results presented in this chapter include the evaluation of the trained YOLO11n model, visual detection outputs, object counting results, stock-status classification, alert generation, database and dashboard operation, and the overall Phase I workflow. The results are based on the implemented system and the available validation data rather than on unimplemented Phase II functionality.

The current YOLO11n model uses a single generic `object` class. Therefore, the results presented in this chapter represent visible retail object detection and counting performance and should not be interpreted as individual brand or SKU recognition accuracy.

7.2 Phase I Implementation Result

The primary outcome of Phase I is a working software-based retail shelf monitoring system. The implemented workflow accepts an uploaded shelf image and processes it through the AI and application pipeline.

The implemented system performs image preprocessing, YOLO11n object detection, confidence filtering, detection processing, visible object counting, shelf-region assignment, rule-based stock evaluation, alert generation, database storage, and dashboard presentation.

The system successfully connects the AI processing layer with the backend application, database, and monitoring interface. This provides a complete software workflow for analysing visible shelf conditions from uploaded images.

Table 7.1 – Phase I Implementation Results

Module	Implemented Result	Outcome

Module	Implemented Result	Outcome
User Authentication	User login and authentication functionality implemented.	Successfully tested
Shelf Image Upload	Shelf images can be uploaded for analysis.	Successfully tested
Image Preprocessing	Uploaded images are prepared for AI inference.	Successfully tested
YOLO11n Detection	Visible retail object instances are detected using YOLO11n.	Successfully tested
Confidence Filtering	Detection results are filtered using the configured confidence threshold.	Successfully tested
Object Counting	Visible detected objects are counted.	Successfully tested
Shelf Region Assignment	Detected objects are processed according to configured shelf regions.	Successfully tested
Stock Evaluation	Detected visible counts are compared with configured minimum thresholds.	Successfully tested
Stock Status	NORMAL, LOW-STOCK, and OUT-OF-STOCK conditions are generated.	Successfully tested
Alert Generation	Applicable monitoring alerts are generated.	Successfully tested
Database Storage	Monitoring information is stored for later retrieval.	Successfully tested
Dashboard	Monitoring results are displayed through the web interface.	Successfully tested
Scan History	Previous monitoring records can be viewed.	Successfully tested

The results in Table 7.1 demonstrate that the major software modules required for Phase I were implemented and integrated successfully.

7.3 YOLO11n Model Evaluation Results

The trained YOLO11n model was evaluated using the validation dataset to measure its ability to detect visible retail object instances. The validation set contained 584 images and 89,934 annotated object instances.

The evaluation was performed using standard object detection metrics including precision, recall, F1-score, mAP@0.5, and mAP@0.5:0.95.

Table 7.2 – YOLO11n Model Evaluation Results

Metric	Result
Precision	76.4%
Recall	68.1%
F1-score	71.9%
mAP@0.5	76.1%
mAP@0.5:0.95	43.7%
Validation Images	584
Validation Instances	89,934
Observed Inference Time	81.1 ms/image

Table 7.2 presents the quantitative evaluation results obtained from the trained YOLO11n model on the validation dataset.

The model achieved a precision of 76.4%, indicating that a substantial proportion of the generated detections corresponded to annotated object instances. The recall of 68.1% indicates that the model detected a considerable proportion of the visible ground-truth object instances in the validation dataset.

The F1-score of 71.9% provides a combined measure of precision and recall. The model achieved an mAP@0.5 of 76.1%, while the stricter mAP@0.5:0.95 value was 43.7%. The difference between these two values indicates that achieving accurate bounding-box localization becomes more challenging when stricter IoU criteria are applied.

The observed inference time was 81.1 ms per image under the CPU-based evaluation environment. This value represents model inference time and does not represent the complete end-to-end response time of the ShelfAI application.

Since the model contains only the generic `object` class, these metrics evaluate generic visible object detection rather than individual product or SKU classification.

7.4 Visual Detection Results

The visual detection output demonstrates the ability of the trained YOLO11n model to identify visible retail object instances within shelf images. The detected objects are represented using bounding boxes generated by the model.

Figure 7.1 – YOLO11n Object Detection

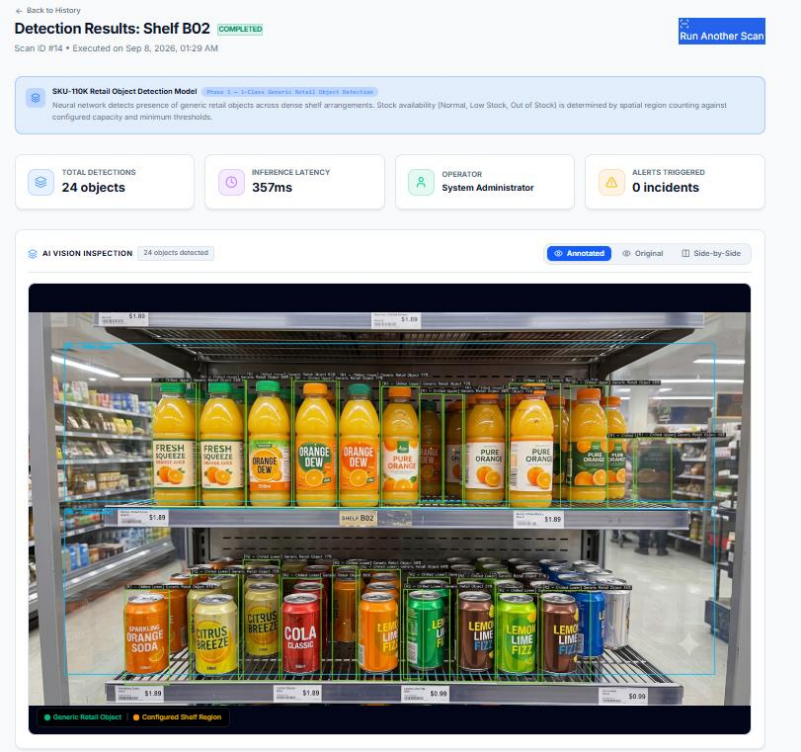


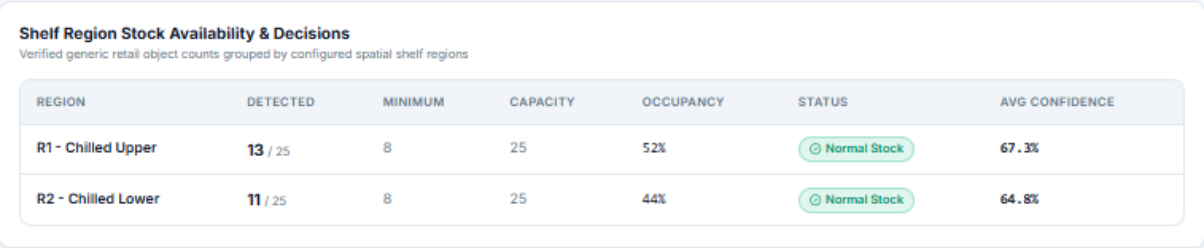
Figure 7.1 shows the bounding boxes generated by YOLO11n for visible retail object instances in a shelf image.

The visual output also demonstrates the basis for the subsequent counting and shelf-region analysis stages. However, the detection image itself is qualitative evidence; the quantitative detection performance is represented by the precision, recall, F1-score, and mAP values presented in Table 7.2.

7.5 Object Counting Results

Following object detection and confidence filtering, the detected bounding boxes are processed to determine the number of visible object instances.

Figure 7.2 – Visible Object Counting Result



Shelf Region Stock Availability & Decisions

Verified generic retail object counts grouped by configured spatial shelf regions

REGION	DETECTED	MINIMUM	CAPACITY	OCCUPANCY	STATUS	AVG CONFIDENCE
R1 - Chilled Upper	13 / 25	8	25	52%	Normal Stock	67.3%
R2 - Chilled Lower	11 / 25	8	25	44%	Normal Stock	64.8%

Figure 7.2 shows the visible object count obtained from the detected shelf objects. For example, an application scan may produce a detected object count such as 47 objects for a particular shelf image. The detected count is used as the input for subsequent shelf-condition evaluation.

The count represents visible detected objects and should not be considered an exact physical inventory quantity because objects may be hidden, partially visible, overlapping, or missed by the detection model.

7.6 Shelf Region and Stock Status Results

The detected objects are processed according to the configured shelf-region information. The visible count associated with the relevant region is then compared with the configured minimum stock threshold.

The stock evaluation follows the defined rules:

where represents the detected visible object count and represents the configured minimum stock threshold.

Figure 7.3 – Shelf Region and Stock Status Result

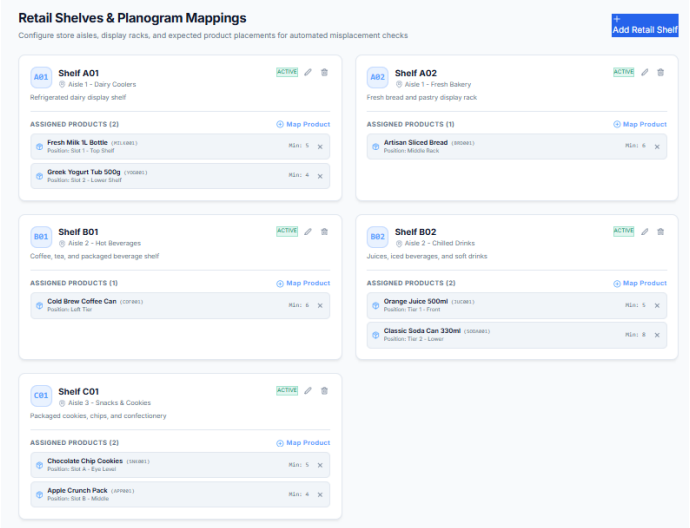


Figure 7.3 shows the configured shelf region and the corresponding stock status generated from the detected visible object count.

The threshold is maintained as application configuration and is not learned by the YOLO11n model.

7.7 Dashboard and Monitoring Results

The ShelfAI dashboard provides a centralized interface for viewing the results produced by the monitoring pipeline.

Figure 7.4 – ShelfAI Dashboard and Monitoring Interface

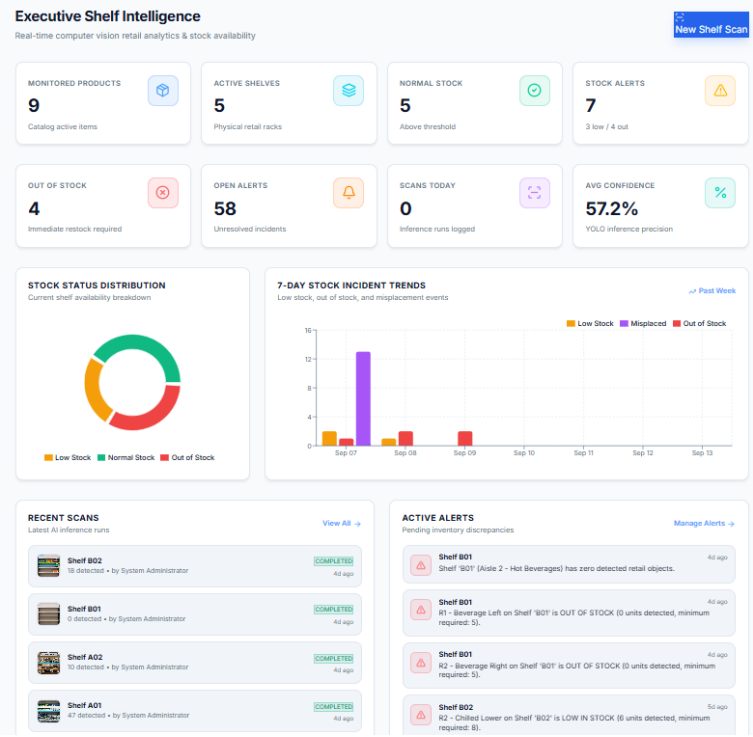


Figure 7.4 shows the dashboard displaying shelf monitoring results and related status information.

The dashboard connects the AI processing results with the user-facing monitoring interface, allowing store staff or managers to interpret the generated shelf condition without directly interacting with the underlying detection model.

7.8 Alert and Monitoring Results

The application generates monitoring alerts when the rule-based evaluation identifies a low-stock or out-of-stock condition.

Figure 7.5 – Shelf Monitoring Alert Result

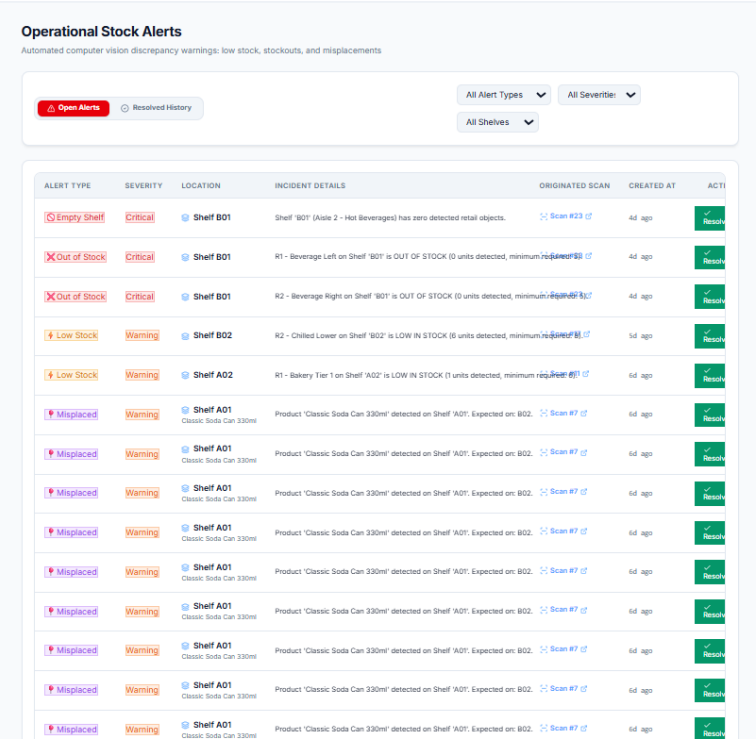


Figure 7.5 shows an alert generated for a shelf condition requiring monitoring attention.

Where shelf or product configuration is used by the application, the alert can be associated with the corresponding configured shelf information. The alert should not be interpreted as evidence of individual SKU recognition because the current YOLO11n model uses the generic `object` class.

7.9 Database and Historical Monitoring Results

The monitoring information generated during shelf analysis is stored in the application database. This provides persistence of scan results and enables historical monitoring.

Figure 7.6 – Historical Monitoring / Scan Results

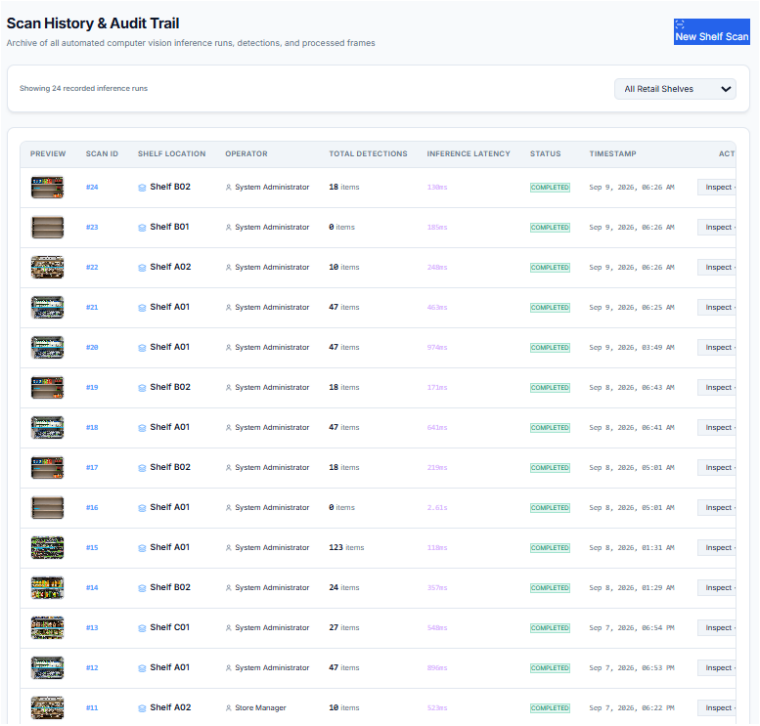


Figure 7.6 shows previously recorded shelf scans and their corresponding monitoring results.

The storage of historical results provides an important connection between individual AI-based image analysis operations and longer-term shelf monitoring.

7.10 Application Scan Results

During application testing, the ShelfAI system successfully processed shelf images and generated detection and monitoring results. The application recorded examples such as a scan containing 47 detected visible objects with an observed processing time of approximately 856 ms and another scan containing 14 detected visible objects with an observed processing time of approximately 811 ms.

These application-level times represent the observed processing time for those scans and should not be confused with the 81.1 ms/image model inference time reported during model validation.

Table 7.3 – Application Monitoring Results

Scan	Detected Visible Objects	Observed Processing Time	Monitoring Result
A01 Scan #10	47	Approximately 856 ms	No alert recorded
A01 Scan #7	14	Approximately 811 ms	Misplacement alerts recorded

Table 7.3 presents selected application-level scan results obtained during ShelfAI testing. The results demonstrate that the application can process different shelf images and generate scan-specific monitoring information. The observed misplacement alerts are based on the application's configured shelf and region logic and should not be interpreted as learned SKU-level recognition by the YOLO11n model.

7.11 Discussion of Results

The Phase I results demonstrate that ShelfAI successfully combines AI-based object detection with application-level shelf monitoring logic. The YOLO11n model provides the visual detection component, while the subsequent processing stages convert the detection output into visible object counts and shelf-level monitoring information.

The precision of 76.4% and mAP@0.5 of 76.1% indicate that the model is capable of detecting visible retail object instances in the selected validation dataset. The recall of 68.1% indicates that some annotated objects are not detected, which is expected to be a challenge in dense shelf environments containing overlapping and partially occluded objects.

The mAP@0.5:0.95 value of 43.7% is lower than the mAP@0.5 value because the stricter metric requires more accurate bounding-box localization across multiple IoU thresholds. This indicates that there remains scope for improving localization accuracy.

The observed inference time of 81.1 ms/image demonstrates that the lightweight YOLO11n model can perform inference on the CPU-based development environment. However, the model inference time and complete application processing time are different measurements. The application-level scan times include additional operations such as image handling, backend processing, database operations, and result presentation.

The system's stock monitoring functionality is based on visible object counts and configured thresholds. Therefore, the resulting stock condition is an image-based availability assessment. It cannot guarantee the exact physical quantity of products on the shelf when objects are hidden or not detected.

The current one-class model is also a significant limitation. It detects generic object instances rather than distinguishing individual brands or SKUs. The configured product names and shelf information in the application therefore serve as monitoring configuration rather than learned product classes.

7.12 Overall Phase I Outcome

The overall Phase I outcome can be summarized as the successful implementation and evaluation of a software-based retail shelf monitoring system. The system accepts shelf images, performs AI-based object detection, counts visible object instances, evaluates shelf conditions using configured thresholds, generates applicable alerts, stores monitoring information, and presents the results through a web-based dashboard.

Table 7.4 – Overall Phase I Result Summary

Evaluation Area	Result
Software architecture	Implemented
Shelf image upload	Successfully implemented and tested
YOLO11n object detection	Successfully implemented and evaluated
Confidence filtering	Successfully implemented and tested
Visible object counting	Successfully implemented and tested
Shelf-region analysis	Successfully implemented and tested
Stock-status evaluation	Successfully implemented and tested
Alert generation	Successfully implemented and tested
Database integration	Successfully implemented and tested
Dashboard	Successfully implemented and tested
Historical monitoring	Successfully implemented and tested

Evaluation Area	Result
Model Precision	76.4%
Model Recall	68.1%
Model F1-score	71.9%
mAP@0.5	76.1%
mAP@0.5:0.95	43.7%
Observed model inference time	81.1 ms/image

The results confirm that the major objectives of the Phase I software implementation were achieved. The system provides a functional foundation for AI-assisted retail shelf monitoring using image-based object detection and rule-based stock evaluation.

7.13 Limitations Identified from the Results

Although the Phase I system achieved the intended software implementation, the results also identify several limitations. The current model detects a generic object class and therefore cannot reliably identify individual products or SKUs. Detection performance may also be affected by dense product arrangements, occlusion, overlapping objects, lighting variations, and image quality.

The visible object count is not equivalent to exact physical inventory because products hidden behind visible objects cannot be reliably counted using a single shelf image. In addition, the stock status depends on configured thresholds and therefore represents an application-level image-based assessment.

These limitations provide the basis for future development involving a project-specific product dataset, SKU-level recognition, improved detection performance, and physical stock verification.

CHAPTER 8

CONCLUSION AND FUTURE ENHANCEMENTS

8.1 Conclusion

The ShelfAI project presents a software-based artificial intelligence approach for retail shelf monitoring and image-based stock availability detection. The Phase I system is designed to assist store staff and managers by analysing uploaded shelf images and converting visual shelf information into meaningful monitoring results.

The implemented system uses the YOLO11n object detection model as its primary artificial intelligence component. The uploaded shelf image is processed through image preprocessing, object detection, confidence filtering, detection processing, visible object counting, shelf-region assignment, and rule-based stock evaluation. The resulting shelf condition is classified as NORMAL, LOW-STOCK, or OUT-OF-STOCK based on the detected visible object count and the configured minimum-stock threshold. Monitoring results and applicable alerts are stored in the database and presented through the web-based dashboard and historical monitoring interface.

The trained YOLO11n model was evaluated using a validation dataset containing 584 images and 89,934 annotated object instances. The model achieved a precision of 76.4%, recall of 68.1%, F1-score of 71.9%, mAP@0.5 of 76.1%, and mAP@0.5:0.95 of 43.7%. The observed model inference time was 81.1 ms per image under the CPU-based evaluation environment. These results demonstrate that the trained model is capable of detecting visible retail object instances in the selected validation dataset.

The Phase I application was also functionally and integratively tested. The implemented workflow, from user authentication and shelf image upload through AI processing, object counting, stock evaluation, alert generation, database storage, and dashboard presentation, was successfully tested.

The current system uses a single generic object class. Therefore, the implemented model performs visible retail object detection rather than reliable individual product, brand, or SKU recognition. Similarly, the stock status generated by the application represents an image-based assessment based on visible detected objects and configured thresholds; it does not provide an exact measurement of the physical quantity of products hidden behind visible items.

Overall, Phase I establishes a functional foundation for AI-assisted retail shelf monitoring. It demonstrates how computer vision and application-level decision rules can be combined to support automated shelf observation and monitoring. The limitations identified during Phase I provide a clear direction for further development of the system.

8.2 Limitations

Although the Phase I system successfully implements the intended software workflow, several limitations remain.

The current YOLO11n model uses a single generic object class. Consequently, the system cannot reliably distinguish individual products, brands, or SKU categories using the current model. Product-specific recognition would require a dedicated dataset containing the target products with appropriate product-level annotations and a corresponding model-training process.

The system also relies on visible objects in a single uploaded image. Products that are hidden behind other products, heavily occluded, partially visible, or affected by unsuitable image conditions may not be detected correctly. Dense shelf arrangements, overlapping products, lighting variations, image quality, and viewing angle can therefore influence detection and counting performance.

The detected object count represents the number of visible object instances identified by the model and does not necessarily represent the exact physical inventory quantity. The current stock status is determined using configured minimum-stock thresholds and therefore provides an image-based shelf availability assessment.

The current Phase I system is based on uploaded images rather than continuous live camera monitoring. The observed inference time of 81.1 ms per image represents model inference under the CPU-based evaluation environment and should not be considered a complete real-time end-to-end application response measurement.

Physical stock verification is another limitation of the current phase. Since ESP32, load cells, and HX711-based sensing are not implemented in Phase I, the system does not currently measure the physical weight of products or compare visual quantities with sensor-derived quantities.

These limitations indicate the areas that require further development to improve the accuracy, reliability, and practical applicability of the ShelfAI system.

8.3 Future Enhancements

The first major future enhancement is the development of a project-specific retail shelf dataset. The dataset should contain the actual products intended for monitoring and should include appropriate product-level or SKU-level annotations. It should represent realistic shelf conditions, including different product orientations, lighting conditions, product arrangements, overlapping objects, partial occlusion, and different shelf viewpoints.

The project-specific dataset can be used to train or fine-tune a suitable object detection model for individual product or SKU recognition. The resulting model can be evaluated using separate training, validation, and testing datasets to determine its product-level detection performance. This enhancement would allow the system to move beyond the current generic object detection capability.

Another major enhancement is the introduction of physical stock verification using IoT hardware. ESP32 can be used as the microcontroller for acquiring measurements from load cells through an HX711 amplifier. The sensing system can be calibrated using known product quantities and controlled shelf conditions to establish the relationship between measured weight and physical product quantity.

The visual and physical measurements can subsequently be compared to identify discrepancies between the detected visible quantity and the physically measured quantity. This can help address situations in which products are hidden or cannot be reliably counted from a shelf image alone. The extended workflow can therefore be represented as:

Project-Specific SKU Recognition → Visual Quantity Estimation → Physical Weight Measurement → Quantity Comparison → Discrepancy Analysis → Improved Stock Verification

A future version of ShelfAI can also support live camera-based monitoring or periodic image acquisition instead of relying only on manually uploaded images. Multiple cameras can be introduced to monitor different shelf sections or store areas.

The system can further be enhanced with improved SKU-level recognition and planogram compliance analysis so that individual products can be distinguished and their placement can be compared with expected shelf arrangements.

The IoT subsystem can be extended with sensor anomaly detection and calibration monitoring to identify unreliable or abnormal physical measurements. Combining visual and physical measurements can improve the reliability of stock verification.

The application can also be extended to support multi-store or cloud-based monitoring, allowing managers to monitor shelf conditions across multiple retail locations through a centralized interface.

Further enhancements may include POS integration, automated replenishment recommendations, mobile notifications, demand forecasting, and intelligent inventory recommendations. These features should be introduced after the core detection, stock verification, and monitoring functions have been experimentally validated.

The future enhancements are intended to extend the current Phase I software foundation into a more comprehensive retail monitoring solution capable of combining visual intelligence with physical stock verification and broader inventory-management functionality.

REFERENCES

[1] A. Franco, D. Maltoni, and S. Papi, “Grocery product detection and recognition,” Expert Systems with Applications, vol. 81, pp. 163–176, 2017, doi: 10.1016/j.eswa.2017.03.018.

[2] E. Goldman, R. Herzig, A. Eisenschtat, J. Goldberger, and T. Hassner, “Precise detection in densely packed scenes,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2019, pp. 5227–5236, doi: 10.1109/CVPR.2019.00537.

[3] X.-S. Wei, Q. Cui, L. Yang, P. Wang, and L. Liu, “RPC: A large-scale retail product checkout dataset,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW), 2019.

[4] Y. Bai, Y. Chen, W. Yu, L. Wang, and W. Zhang, “Products-10K: A large-scale product recognition dataset,” arXiv:2008.10545, 2020.

[5] R. Pietrini, L. Rossi, A. Mancini, P. Zingaretti, E. Frontoni, and M. Paolanti, “A deep learning-based system for product recognition in intelligent retail environment,” in Image Analysis and Processing – ICIAP 2021 Workshops, Lecture Notes in Computer Science, vol. 13232, Springer, 2022, pp. 371–382, doi: 10.1007/978-3-031-06430-2_31.

[6] “Advances in deep learning-based image recognition of product packaging,” Image and Vision Computing, vol. 128, Art. no. 104571, 2022, doi: 10.1016/j.imavis.2022.104571.

[7] B. Santra, U. Ghosh, and D. P. Mukherjee, “Graph-based modelling of superpixels for automatic identification of empty shelves in supermarkets,” Pattern Recognition, vol. 127, Art. no. 108627, 2022, doi: 10.1016/j.patcog.2022.108627.

[8] “Enhanced out-of-stock detection in retail shelf images based on deep learning,” Sensors, vol. 24, no. 2, Art. no. 693, 2024, doi: 10.3390/s24020693.

[9] D. Allegra, M. Litrico, M. A. N. Spatafora, F. Stanco, and G. M. Farinella, “Exploiting egocentric vision on shopping cart for out-of-stock detection in retail environments,” in Proc. IEEE/CVF Int. Conf. Computer Vision Workshops (ICCVW), 2021, pp. 1735–1740, doi: 10.1109/ICCVW54120.2021.00199.

[10] G. De Simone, P. Foggia, A. Saggese, and M. Vento, “Autonomous mobile robot for automatic out of stock detection in a supermarket,” in Proc. IEEE/CVF Int. Conf. Computer Vision Workshops (ICCVW), 2023, pp. 1829–1838.

[11] M. Saqlain et al., “Hybrid approach for shelf monitoring and planogram compliance (Hyb-SMPC) in retails using deep learning and computer vision,” Mathematical Problems in Engineering, 2022, Art. no. 4916818, doi: 10.1155/2022/4916818.

[12] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 39, no. 6, pp. 1137–1149, 2017, doi: 10.1109/TPAMI.2016.2577031.

[13] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal speed and accuracy of object detection,” arXiv:2004.10934, 2020.

[14] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2023, pp. 7464–7475.

[15] B. Santra, A. K. Shaw, and D. P. Mukherjee, “Part-based annotation-free fine-grained classification of images of retail products,” Pattern Recognition, vol. 121, Art. no. 108257, 2022, doi: 10.1016/j.patcog.2021.108257.

[16] “Generating training images with different angles by GAN for improving grocery product image recognition,” Neurocomputing, vol. 488, pp. 694–705, 2022, doi: 10.1016/j.neucom.2021.11.080.